Kubernetes - Orchestration de Conteneurs

🎯 Utilité dans le projet

Kubernetes (K8s) **orchestre et gère** les conteneurs Docker en production :

- 🔄 Auto-scaling : Ajuste le nombre de pods selon la charge
- 💾 Self-healing : Redémarre les pods défaillants
- ⚖️ Load balancing : Distribue le trafic
- 🔄 Rolling updates : Déploiement sans downtime
- 📦 Service discovery : DNS interne automatique

Architecture du projet

```
│  │  mongo-pvc              │                        │
│  │  1Gi PersistentVolume   │                        │
│  └─────────────────────────┘                        │
│                                                     │
└─────────────────────────────────────────────────────┘
```

📦 Objets Kubernetes déployés

1. Namespace

```yaml
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: devops-portfolio
  labels:
    name: devops-portfolio
```

Rôle : Isole les ressources du projet

2. Secret

```yaml
# k8s/secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mern-secret
  namespace: devops-portfolio
type: Opaque
data:
  JWT_SECRET: <base64>
  MONGO_PASSWORD: <base64>
  MONGO_URI: <base64>
```

Rôle : Stocke les credentials de manière sécurisée

3. ConfigMap

```yaml
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: mern-config
  namespace: devops-portfolio
data:
  NODE_ENV: "production"
  PORT: "5000"
  VITE_API_URL: "/api"
```

Rôle : Configuration non sensible

4. MongoDB Deployment

```yaml
# k8s/mongo/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-deployment
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: mongodb
        image: mongo:7
        ports:
        - containerPort: 27017
        volumeMounts:
        - name: mongo-data
          mountPath: /data/db
      volumes:
      - name: mongo-data
        persistentVolumeClaim:
          claimName: mongo-pvc
```

Caractéristiques :

- 1 replica (stateful)
- Volume persistant
- Health checks

5. MongoDB PVC

```yaml
# k8s/mongo/pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Rôle : Demande de stockage persistant

6. MongoDB Service

```yaml
# k8s/mongo/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  type: ClusterIP
  selector:
    app: mongo
  ports:
  - port: 27017
    targetPort: 27017
```

Type ClusterIP : Accessible uniquement depuis le cluster

7. Backend Deployment

```yaml
# k8s/backend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: backend
        image: lims4/devops-portfolio-mern-backend:latest
        imagePullPolicy: Always
        ports:
        - containerPort: 5000
        env:
        - name: JWT_SECRET
          valueFrom:
            secretKeyRef:
              name: mern-secret
              key: JWT_SECRET
        livenessProbe:
          httpGet:
            path: /api/health
            port: 5000
          initialDelaySeconds: 30
```

Caractéristiques :

- Image depuis Docker Hub
- Variables d'environnement depuis Secret/ConfigMap
- Health checks HTTP

8. Backend Service

```yaml
# k8s/backend/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
  - port: 5000
    targetPort: 5000
```

9. Frontend Deployment

```
# k8s/frontend/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
spec:
  replicas: 1
  template:
    spec:
      containers:
      - name: frontend
        image: lims4/devops-portfolio-mern-frontend:latest
        ports:
        - containerPort: 80
```

10. Frontend Service

```
# k8s/frontend/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Type NodePort : Accessible depuis l'extérieur sur le port 30080

🚀 Déploiement

Méthode 1 : kubectl (manuel)

```
# 1. Créer le namespace
kubectl apply -f k8s/namespace.yaml

# 2. Secrets et ConfigMap
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/configmap.yaml

# 3. MongoDB
kubectl apply -f k8s/mongo/

# 4. Backend
kubectl apply -f k8s/backend/

# 5. Frontend
kubectl apply -f k8s/frontend/

# 6. Vérifier
kubectl get all -n devops-portfolio
```

Méthode 2 : Jenkins (automatique)

```
stage('Deploy to Kubernetes') {
    sh """
        kubectl apply -f k8s/
        kubectl rollout restart deployment/backend-deployment
        kubectl rollout status deployment/backend-deployment
    """
}
```

🔍 Commandes essentielles

Voir les ressources

```
# Tous les objets
kubectl get all -n devops-portfolio

# Pods avec détails
kubectl get pods -n devops-portfolio -o wide

# Services
kubectl get svc -n devops-portfolio

# Deployments
kubectl get deployments -n devops-portfolio
```

Logs

```
# Logs d'un pod
kubectl logs -f <pod-name> -n devops-portfolio

# Logs d'un deployment
kubectl logs -f deployment/backend-deployment -n devops-portfolio

# Logs depuis le début
kubectl logs <pod-name> -n devops-portfolio --since=1h
```

Debug

```
# Describe (événements, erreurs)
kubectl describe pod <pod-name> -n devops-portfolio

# Entrer dans un pod
kubectl exec -it <pod-name> -n devops-portfolio -- sh

# Port-forward local
kubectl port-forward svc/frontend-service 3000:80 -n devops-portfolio
```

Scaling

```
# Scaler manuellement
kubectl scale deployment backend-deployment --replicas=3 -n devops-portfolio

# Auto-scaling (HPA - nécessite metrics-server)
kubectl autoscale deployment backend-deployment \
  --min=2 --max=5 --cpu-percent=80 \
  -n devops-portfolio
```

Rolling update

```
# Mettre à jour l'image
kubectl set image deployment/backend-deployment \
  backend=lims4/backend:v2.0.0 \
  -n devops-portfolio

# Voir le statut du rollout
kubectl rollout status deployment/backend-deployment -n devops-portfolio

# Rollback
kubectl rollout undo deployment/backend-deployment -n devops-portfolio
```

🏥 Self-healing

Kubernetes redémarre automatiquement les pods défaillants :

```
# Simuler un crash
kubectl delete pod <pod-name> -n devops-portfolio

# Observer la recréation automatique
kubectl get pods -n devops-portfolio -w
```

📊 État actuel du projet

```
$ kubectl get all -n devops-portfolio

NAME                                  READY   STATUS    RESTARTS
pod/mongo-deployment-xxx              1/1     Running   0
pod/backend-deployment-xxx            1/1     Running   0
pod/frontend-deployment-xxx           1/1     Running   0

NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
service/mongo-service      ClusterIP   10.x.x.x        <none>        27017/TCP
service/backend-service    ClusterIP   10.x.x.x        <none>        5000/TCP
service/frontend-service   NodePort    10.x.x.x        <none>        80:30080/TCP

NAME                                 READY   UP-TO-DATE   AVAILABLE
deployment.apps/mongo-deployment     1/1     1            1
deployment.apps/backend-deployment   1/1     1            1
deployment.apps/frontend-deployment  1/1     1            1
```

🔒 RBAC - Jenkins deployer

Pour permettre à Jenkins de déployer :

```yaml
# k8s/jenkins-rbac.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-deployer
  namespace: devops-portfolio
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-deploy-role
  namespace: devops-portfolio
rules:
- apiGroups: ["apps", "", "batch"]
  resources: ["deployments", "services", "pods", "jobs"]
  verbs: ["get", "list", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-deploy-binding
  namespace: devops-portfolio
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: jenkins-deploy-role
subjects:
- kind: ServiceAccount
  name: jenkins-deployer
  namespace: devops-portfolio
```

💡 Bonnes pratiques appliquées

- ✅ **Namespace** : Isolation des ressources
- ✅ **Labels** : Organisation et sélection
- ✅ **Health checks** : Détection de défaillances
- ✅ **Resource limits** : Éviter la surcharge
- ✅ **ConfigMap/Secret** : Séparation config/code
- ✅ **Services** : Load balancing automatique
- ✅ **PVC** : Persistence des données

Prochaine étape : 05-TERRAFORM.md